

```

/*
 * assembly.s
 *
 */

@ DO NOT EDIT
    .syntax unified
    .text
    .global ASM_Main
    .thumb_func

@ DO NOT EDIT
vectors:
    .word 0x20002000
    .word ASM_Main + 1

@ DO NOT EDIT label ASM_Main
ASM_Main:

    @ Some code is given below for you to start with
    LDR R0, RCC_BASE                @ Enable clock for GPIOA and B by setting
bit 17 and 18 in RCC_AHBENR
    LDR R1, [R0, #0x14]
    LDR R2, AHBENR_GPIOAB          @ AHBENR_GPIOAB is defined under LITERALS at the end
of the code
    ORRS R1, R1, R2
    STR R1, [R0, #0x14]

    LDR R0, GPIOA_BASE              @ Enable pull-up resistors for pushbuttons
    MOVS R1, #0b01010101
    STR R1, [R0, #0x0C]
    LDR R1, GPIOB_BASE              @ Set pins connected to LEDs to outputs
    LDR R2, MODER_OUTPUT
    STR R2, [R1, #0]
    MOVS R2, #0                    @ NOTE: R2 will be dedicated to holding the value on
the LEDs
    MOVS R3, #1                    @ default increment = 1
    LDR R4, LONG_DELAY_CNT          @ default delay = long (0.7s)

@ TODO: Add code, labels and logic for button checks and LED patterns

main_loop:
    @ --- Check button states ---
    LDR R5, GPIOA_BASE
    LDR R6, [R5, #0x10]            @ IDR register (input data register)

    @ Buttons are pull-up → pressed = 0 (bit = 0 when pressed)
    @ Need to check each button individually

    @ Check SW0 (PA0) - increment by 2 when pressed

```

```

    MOVS R7, #0x01          @ Load mask for SW0 (PA0)
    TST R6, R7              @ Test bit 0 (SW0)
    BNE check_sw1          @ If not pressed, check next button
    MOVS R3, #2             @ Set increment to 2
    LDR R4, LONG_DELAY_CNT  @ Use long delay (0.7s)
    B check_sw2             @ Continue checking other buttons

```

check_sw1:

```

    @ If SW0 not pressed, revert to default increment of 1
    MOVS R3, #1            @ Default increment = 1
    LDR R4, LONG_DELAY_CNT @ Default delay = long (0.7s)

```

@ Check SW1 (PA1) - faster timing when pressed

```

    MOVS R7, #0x02          @ Load mask for SW1 (PA1)
    TST R6, R7              @ Test bit 1 (SW1)
    BNE check_sw2          @ If not pressed, check next button
    LDR R4, SHORT_DELAY_CNT @ Use short delay (0.3s)

```

check_sw2:

@ Check SW2 (PA2) - set pattern to 0xAA when pressed

```

    MOVS R7, #0x04          @ Load mask for SW2 (PA2)
    TST R6, R7              @ Test bit 2 (SW2)
    BNE check_sw3          @ If not pressed, check next button
    MOVS R2, #0xAA          @ Set LED pattern to 0xAA
    B write_leds            @ Skip increment and go to display

```

check_sw3:

@ Check SW3 (PA3) - freeze pattern when pressed

```

    MOVS R7, #0x08          @ Load mask for SW3 (PA3)
    TST R6, R7              @ Test bit 3 (SW3)
    BNE buttons_done        @ If not pressed, continue normally
    B write_leds            @ Freeze - skip increment

```

buttons_done:

@ --- Update LED pattern normally ---

```

    ADDS R2, R2, R3          @ Increment LED pattern

```

write_leds:

@ --- Delay ---

```

    MOV R7, R4               @ Load delay counter

```

delay_loop:

```

    SUBS R7, R7, #1
    BNE delay_loop

```

@ --- Write to LEDs ---

```

    STR R2, [R1, #0x14]      @ Output to GPIOB_ODR (LEDs)
    B main_loop

```

@ LITERALS; DO NOT EDIT

```

        .align
RCC_BASE:                .word 0x40021000
AHBENR_GPIOAB:          .word 0b11000000000000000000
GPIOA_BASE:              .word 0x48000000
GPIOB_BASE:              .word 0x48000400
MODER_OUTPUT:            .word 0x5555

@ TODO: Add your own values for these delays
        @ default increment 1 every 0.7
        @SW0 increment 2 evry 0.7
        @SW1 increment 1 every 0.3
        @SW2 pattern 0xAA
        @SW3 stop pattern
LONG_DELAY_CNT:           .word 700000    @ Adjust for 0.7s delay
SHORT_DELAY_CNT:          .word 300000    @ Adjust for 0.3s delay

```